



Università degli Studi dell'Insubria

A.A. 2025/2026

Laboratorio Interdisciplinare A



CineMax
All'Avanguardia del Cinema

MANUALE TECNICO

Docente:

Prof. Giovanni Meroni

Versione del Documento:

v1.0

Autori:

Casagrande Giulia Alessandra

Consiglio Matteo

Della Torre Edoardo

Ferrigno Valeria

INDICE

INDICE	1
INTRODUZIONE.....	Errore. Il segnalibro non è definito.
INFRASTRUTTURA E INSTALLAZIONE	Errore. Il segnalibro non è definito.
Requisiti di Sistema.....	Errore. Il segnalibro non è definito.
Setup dell'Ambiente.....	Errore. Il segnalibro non è definito.
Installazione del Programma	Errore. Il segnalibro non è definito.
ESECUZIONE E GUIDA ALL'USO.....	Errore. Il segnalibro non è definito.
Uso delle Funzionalità	Errore. Il segnalibro non è definito.
Menù principale e accesso ospite	Errore. Il segnalibro non è definito.
Menu Cliente (Previa autenticazione).....	Errore. Il segnalibro non è definito.
Menu Proiezionista (Gestore del Palinsesto).....	Errore. Il segnalibro non è definito.
Menu Bigliettaio (Operatore di Cassa).....	Errore. Il segnalibro non è definito.
Dataset di Test.....	Errore. Il segnalibro non è definito.
Controllo eccezioni	Errore. Il segnalibro non è definito.
LIMITI DELLA SOLUZIONE SVILUPPATA.....	Errore. Il segnalibro non è definito.
SITOGRAFIA E BIBLIOGRAFIA.....	Errore. Il segnalibro non è definito.

INTRODUZIONE E SPECIFICHE TECNICHE

Il presente manuale tecnico descrive in dettaglio la progettazione e l'implementazione del software **CineMax**, un sistema Object-Oriented finalizzato alla gestione del flusso operativo di una struttura cinematografica monosala.

L'intero sistema è stato ingegnerizzato per garantire la massima portabilità e aderenza alle specifiche accademiche del corso. Le specifiche tecniche di riferimento sono:

- **Linguaggio di Programmazione:** Java (compatibile con Java SE 8 e versioni successive).
- **Metodo di Programmazione:** Programmazione a Oggetti (OOP) con incapsulamento rigido dei dati, ereditarietà per la specializzazione dei ruoli e polimorfismo dinamico per la gestione dei flussi utente.
- **Nessuna libreria esterna:** Il software si affida esclusivamente alle librerie native del Java Development Kit (JDK), nello specifico ai package `java.io`, `java.util` e `java.time`.
- **Persistenza:** Implementazione di un sistema di salvataggio file locale mediante tre file piatti in formato strutturato *Comma-Separated Values* (CSV).

Nel corso di questo manuale, si andranno ad analizzare tutti gli aspetti tecnici del software.

ARCHITETTURA DEL SISTEMA E MODELLO DELLE CLASSI

Schema delle Relazioni e Gerarchia Ereditaria

L'architettura del software si basa sulla separazione tra la logica di controllo guidata dall'interfaccia utente (MainCinemax), il controllore dei dati e della persistenza (Manager), e le classi rappresentanti le entità del dominio di business.

La gestione dell'anagrafica e dei permessi si sviluppa su una gerarchia ereditaria rigorosa, la cui radice è rappresentata dalla classe astratta Utente. Questa struttura consente di centralizzare i dati comuni (nome, cognome, credenziali) ed estendere i comportamenti specifici tramite polimorfismo.

Descrizione Puntuale dei Moduli del Dominio

- **MainCinemax:** Rappresenta l'entry point dell'applicazione. Non memorizza lo stato delle entità, ma gestisce i menu testuali della CLI (Command Line Interface). Gestisce lo stato temporaneo della sessione dell'utente correntemente autenticato (currentUser) e i tentativi falliti di accesso.
- **Manager:** È il nucleo logico e operativo del sistema. Contiene le liste in memoria (nella cartella data) di tutte le proiezioni e gli utenti caricati, e incapsula i metodi I/O per la lettura e scrittura dei file CSV.
- **Utente (Classe Astratta):** Definisce gli attributi strutturali di un account e implementa il metodo per verificare la validità temporale di una sessione di lavoro.
- **Cliente:** Estende Utente. Introduce una struttura dati interna (LinkedList<Prenotazione>) per tracciare lo storico delle prenotazioni associate allo specifico utente e i metodi logici per l'acquisto o la cancellazione dei biglietti.
- **Proiezionista:** Estende Utente. Fornisce l'accesso ai metodi di scrittura del palinsesto (creazione e rimozione di oggetti Proiezione) e al calcolo analitico delle statistiche occupazionali della sala.
- **Bigliettaio:** Estende Utente. Fornisce l'accesso alle funzioni di cassa fisica, abilitando la vendita diretta e la manipolazione delle prenotazioni di qualsiasi cliente tramite l'identificativo univoco (ID).
- **Proiezione:** Modella un evento cinematografico. Associa un oggetto Film a una specifica data e ora, definisce il prezzo del biglietto e incapsula la matrice bidimensionale dei posti.
- **Film:** Modello dati anagrafico puro che descrive le caratteristiche stabili di una pellicola (titolo, regista, genere, durata, restrizioni sull'età).

- **Posto:** Identifica una singola coordinata in sala tramite la codifica di una fila testuale (convertita internamente in un indice numerico a base 1) e del relativo numero di colonna.
- **Prenotazione:** Associa un codice identificativo incrementale statico (idCounter) a un oggetto Proiezione, memorizzando la lista dei posti prenotati e la spesa complessiva sostenuta.
- **Date e Time:** Classi custom adibite alla gestione dell'informazione temporale. Sostituiscono e isolano le complessità dei calendari standard tramite un sistema di validazione interno e metodi di calcolo cronologico.
- **Ruolo (Enum):** Enumerazione che mappa i ruoli supportati dall'applicativo (UTENTE, CLIENTE, PROIEZIONISTA, BIGLIETTAIO), definendo per ciascuno di essi i relativi livelli di autorizzazione amministrativa (power).

Dizionario dettagliato delle classi e delle interfacce

Di seguito viene censito il dizionario delle classi costituenti il nucleo operativo del package cinemax. Per ciascun elemento vengono dettagliati il livello di incapsulamento, i campi di stato e le responsabilità logiche espresse tramite i metodi.

Classe Astratta: Utente

Rappresenta lo scheletro strutturale per la profilazione degli account operanti sul sistema.

- **Modificatori d'accesso:** public abstract
- **Attributi protetti (protected):**
 - nome (String): Nome anagrafico dell'utente.
 - cognome (String): Cognome anagrafico dell'utente.
 - username (String): Identificativo alfanumerico univoco per il login.
 - password (String): Stringa cifrata tramite algoritmo di Cesare.
 - dataNascita (Date): Oggetto custom per la verifica dell'età minima.
 - luogo (String): Comune di nascita.
 - ruolo (Ruolo): Costante dell'omonima enumerazione.
 - isAdmin (boolean): Flag per l'abilitazione dei menu di amministrazione.
 - isLoggedIn (boolean): Stato di attivazione della sessione di lavoro.

- **Metodi di rilievo:**

- `public boolean login(String username, String password)`: Esegue il match delle credenziali convertendo la stringa in chiaro tramite `SecurityUtils`.
- `public void logout()`: Commuta il flag `isLogged` a `false`.
- `public boolean isSessionValid()`: Verifica che il ciclo vitale della sessione non sia compromesso.

Classe: Cliente

Modella l'attore finale abilitato all'acquisto dei titoli d'accesso ai film.

- **Modificatori d'accesso:** `public` (estende `Utente`)
- **Attributi privati:**
 - `bookings (LinkedList<Prenotazione>)`: Lista concatenata delle prenotazioni attive e storiche del cliente.
- **Metodi di rilievo:**
 - `public void prenota(Proiezione p, LinkedList<Posto> posti)`: Istanza un oggetto `Prenotazione` e ne esegue l'ancoraggio alla lista se i posti superano la validazione.
 - `public void cancellaPrenotazione(Prenotazione pr)`: Esegue lo storno dei posti sulla matrice della proiezione e rimuove l'istanza dalla collezione.

Classe: Manager

Classe di controllo globale (*Controller*) preposta al coordinamento del database in memoria e delle transazioni su file system.

- **Modificatori d'accesso:** `public`
- **Attributi privati statici finali:**
 - `PROIEZIONISTA_FILE, UTENTI_FILE, PRENOTAZIONI_FILE (String)`: Percorsi relativi costanti delle sorgenti CSV.
- **Attributi di istanza:**
 - `proiezioni (LinkedList<Proiezione>)`: Catalogo globale dei film pianificati.
- **Metodi di rilievo:**
 - `public void caricaDati()`: Inizializza i flussi I/O leggendo i file sequenzialmente all'avvio.
 - `public void salvaDati()`: Sincronizza lo stato in-memory sovrascrivendo i file CSV.

STRUTTURE DATI E COERENZA DEI DATI IN MEMORIA

Il software esclude l'uso di database relazionali esterni, gestendo lo stato delle entità interamente all'interno della memoria RAM allocata dalla Java Virtual Machine (JVM). La struttura dati d'elezione per la gestione delle collezioni dinamiche è la `java.util.LinkedList`.

La scelta della lista doppiamente concatenata è motivata dalle specifiche esigenze operative del sistema:

1. **Frequenti Inserimenti e Rimozioni in Testa/Coda:** L'aggiunta di nuove prenotazioni o la cancellazione di proiezioni avvengono con complessità computazionale costante $O(1)$ rispetto all'indirizzamento degli elementi.
2. **Scansione Sequenziale:** Poiché le ricerche avvengono prevalentemente tramite iterazione totale filtrata (es. ricerca per titolo del film o per username), la mancanza di accesso casuale indicizzato tipica dell'`ArrayList` non penalizza le performance complessive nel contesto d'uso di un cinema monosala.

La coerenza dei dati della sala è invece garantita da una matrice bidimensionale di booleani di dimensioni fisse allocata all'interno di ogni singola istanza della classe `Proiezione`:

```
protected boolean[][] sala = new boolean[10][20];
```

La matrice mappa rigidamente 10 file (da 'A' a 'J') e 20 colonne (da 1 a 20). Il valore primitivo `false` denota un posto libero, mentre il valore `true` ne sancisce l'occupazione irreversibile per quella specifica proiezione, impedendo all'origine casi di double-booking.

DETTAGLIO DEGLI ALGORITMI CHIAVE

Algoritmo di Sicurezza e Cifratura delle Password (Cifrario di Cesare)

Per garantire la conformità ai requisiti di riservatezza senza introdurre dipendenze da framework crittografici esterni, la classe utility `SecurityUtils` implementa una variante dell'algoritmo del Cifrario di Cesare con un fattore di spostamento (*shift*) costante pari a 3.

L'algoritmo elabora la stringa carattere per carattere, preservando i caratteri speciali e applicando un'aritmetica modulare separata per le lettere maiuscole (modulo 26), minuscole (modulo 26) e per le cifre numeriche (modulo 10).

```
public static String hashPassword(String password) {
    StringBuilder encrypted = new StringBuilder();

    for (char c : password.toCharArray()) {
        if (Character.isLetter(c)) {
            char base = Character.isUpperCase(c) ? 'A' : 'a';
            int offset = c - base;
            int newOffset = (offset + SHIFT) % 26;
            encrypted.append((char) (base + newOffset));
        } else if (Character.isDigit(c)) {
            int digit = c - '0';
            int newDigit = (digit + SHIFT) % 10;
            encrypted.append((char) ('0' + newDigit));
        } else {
            // Caratteri speciali rimangono uguali
            encrypted.append(c);
        }
    }

    return encrypted.toString();
}
```


Algoritmo di Parsing dei File CSV con Gestione delle Stringhe Complesse

Un limite evidente dei parser CSV elementari basati sul metodo `String.split(",")` consiste nel fallimento sistematico qualora un campo testuale (come il titolo di un film) contenga al proprio interno la virgola come carattere di punteggiatura (es. *"The Good, the Bad and the Ugly"*).

Per superare questo limite, la classe `Manager` implementa un algoritmo di parsing custom basato su una macchina a stati finiti elementare guidata dal flag booleano `insideQuotes`.

```
private String[] splitCsv(String line) {
    LinkedList<String> field = new LinkedList<>();
    StringBuilder current = new StringBuilder();
    boolean insideQuotes = false;

    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (c == '"') {
            insideQuotes = !insideQuotes;
        } else if (c == ',' && !insideQuotes) {
            field.add(current.toString());
            current.setLength(newLength: 0);
        } else {
            current.append(c);
        }
    }

    field.add(current.toString());
    String[] array = new String[field.size()];
    field.toArray(array);
    return array;
}
```

Algoritmo di Rilevamento delle Sovrapposizioni nel Palinsesto Orario

La pianificazione di una nuova proiezione da parte del Proiezionista richiede la verifica dell'assenza di conflitti d'orario all'interno della medesima sala. L'algoritmo converte l'orario di inizio di ogni film in minuti assoluti dall'inizio della giornata ($\text{Ore} \times 60 + \text{minuti}$) e calcola l'orario di fine aggiungendo la durata della pellicola espressa dal modello Film.

Dati due spettacoli programmati nella stessa data, la condizione di sovrapposizione si verifica se l'intervallo temporale del nuovo film $[\text{Inizio}_A, \text{Fine}_A]$ interseca l'intervallo di un film già esistente $[\text{Inizio}_B, \text{Fine}_B]$. La formula logica implementata per rilevare il conflitto è:

$$[\text{Inizio}_A, \text{Inizio}_B] < [\text{Fine}_A, \text{Fine}_B]$$

Se questa condizione si valuta come vera, il metodo blocca l'inserimento sollevando un alert a terminale ed evitando la corruzione logica del palinsesto.

GESTIONE DELLA PERSISTENZA SU FILE

La persistenza dello stato del sistema tra diverse esecuzioni del programma è affidata a tre file posizionati all'interno della directory `data/`: `utenti.csv`, `proiezioni.csv` e `prenotazioni.csv`.

Il caricamento avviene all'avvio dell'applicazione tramite flussi di lettura basati su `BufferedReader` combinati con `FileReader`. La scrittura e l'aggiornamento dei file avvengono in modalità distruttiva o in modalità *append* a seconda del contesto d'uso, sfruttando le classi `BufferedWriter` e `FileWriter`.

Esempio di scrittura in *append* controllata (creazione prenotazione nel file)

```
try (BufferedWriter writer = new BufferedWriter(new
FileWriter(PRENOTAZIONI_FILE, true))) {
    writer.write(strutturaCampiCsv);
    writer.newLine();
} catch (IOException e) {
    System.out.println("Errore critico durante il salvataggio su file: " +
e.getMessage());
}
```

I formati tracciati all'interno dei file di persistenza sono strutturati rigidamente secondo le seguenti testate (*header*):

1. **utenti.csv:**
nome, cognome, username, password, giorno, mese, anno, luogo, ruolo
2. **proiezioni.csv:**
data_ora_proiezione, titolo_film, genere, regista, anno, durata_minuti, eta_minima, prezzo_biglietto
3. **prenotazioni.csv:** id, nome, cognome, username, film, gd, md, ad, ora, qta

Tracciato dei File CSV

Per garantire la manutenibilità del software, viene formalizzata la struttura geometrica dei record memorizzati nei file piatti. Tutti i campi testuali che includono spazi o caratteri speciali sono delimitati da doppi apici ("), mentre il separatore di campo è la virgola (, - ASCII 0x2C).

Tabella dei Tipi e Vincoli di Colonna

1. File utenti.csv

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
nome	Alfanumerico	String	Nome del soggetto censito.
cognome	Alfanumerico	String	Cognome del soggetto censito.
username	Alfanumerico	String [CHIAVE]	Identificativo univoco di autenticazione.
password	Alfanumerico	String (Cifrata)	Stringa elaborata con shift +3.
giorno	Intero	Range: 1 - 31	Giorno della data di nascita.

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
meze	Intero	Range: 1 - 12	Mese della data di nascita.
anno	Intero	Formato AAAA	Anno di nascita (es. 2006).
luogo	Alfanumerico	String	Luogo o comune di nascita.
ruolo	Enumerativo	CLIENTE, PROIEZIONISTA, BIGLIETTAIO	Profilo dei permessi.

2. File proiezioni.csv

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
data_ora_proiezione	Temporale	AAAA-MM-GG HH:MM:SS	Timestamp di inizio dell'evento.
titolo_film	Alfanumerico	String (inclusi doppi apici)	Nome commerciale dell'opera.
genere	Alfanumerico	String	Genere (es. Drama, Biography).
regista	Alfanumerico	String	Nome e cognome del regista.
anno	Intero	Formato AAAA	Anno di produzione cinematografica.

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
durata_minuti	Intero	Valore positivo in minuti	Tempo di proiezione effettivo.
eta_minima	Intero	Es. 0, 12, 14, 18	Vincolo anagrafico per l'accesso.
prezzo_biglietto	Decimale	Float / Double (es. 8.50)	Tariffa monetaria espresso in Euro.

3. File prenotazioni.csv

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
id	Intero	Progressivo univoco	Identificativo numerico della prenotazione (allineato tramite ensureIdCounterAtLeast).
nome	Alfanumerico	String	Nome del titolare della prenotazione.
cognome	Alfanumerico	String	Cognome del titolare della prenotazione.
username	Alfanumerico	String [CHIAVE ESTERNA]	Username del cliente (corrispondente al campo in utenti.csv).
film	Alfanumerico	String (incluso tra doppi apici)	Titolo esatto della pellicola prenotata (chiave di sottomatch).
gd	Intero	Range: 1 - 31	Giorno della data di proiezione dello spettacolo.

Nome Campo	Tipo di Dato	Formato / Vincolo	Descrizione
md	Intero	Range: 1 - 12	Mese della data di proiezione dello spettacolo.
ad	Intero	Formato AAAA	Anno della data di proiezione dello spettacolo.
ora	Temporale	Formato HH:MM	Orario di inizio della proiezione specifica.
qta	Intero	Valore > 0	Quantità complessiva di posti occupati all'interno della medesima transazione.

ANALISI DEI LIMITI E COMPLESSITÀ COMPUTAZIONALE

L'architettura attuale, pur rispondendo pienamente ai requisiti di stabilità, presenta dei vincoli intrinseci di complessità legati all'assenza di un motore database indicizzato:

- **Ricerca degli elementi $O(N)$:** Non esistendo indici hash o strutture ad albero, la localizzazione di un utente per username o di una proiezione per titolo richiede una scansione lineare sequenziale di tutta la LinkedList. Sebbene l'impatto sia trascurabile per l'applicazione monosala ($N < 1000$), le performance degraderebbero linearmente all'aumentare dei volumi storici dei dati.
- **Operazioni di I/O sincrone:** Ogni operazione di salvataggio accede direttamente al file system in modo sincrono sul thread principale dell'applicazione, bloccando momentaneamente l'interfaccia a riga di comando durante le operazioni di scrittura su disco.
- **Mancanza di transazionalità standard:** Se la JVM viene interrotta brutalmente durante la riscrittura di un file CSV, non esistono meccanismi di *rollback* o file di log delle transazioni. Questo espone il sistema a un rischio potenziale di corruzione parziale dei dati in caso di blackout o terminazione forzata del processo d'esecuzione.

POLITICA DI GESTIONE DELLE ECCEZIONI E ROBUSTEZZA

La tolleranza ai guasti e la robustezza dell'applicativo CineMax sono garantite da una strategia duale di gestione delle anomalie: l'utilizzo di eccezioni custom per la logica di business e l'intercettazione localizzata delle eccezioni di sistema per i flussi I/O e di parsing.

Gerarchia delle Eccezioni Custom

Per isolare e identificare gli errori derivanti da formati di input non conformi da parte dell'utente, sono state introdotte due classi di eccezione specifiche:

1. **DateFormatException**: Estende RuntimeException (eccezione *unchecked*). Viene sollevata dai costruttori e dai metodi di validazione della classe Date qualora i valori numerici di giorno, mese o anno non rispettino i confini del calendario gregoriano (es. 31 Novembre o mesi superiori a 12).

```
package cinemax;

/**
 * Eccezione lanciata quando una data non rispetta il formato o i limiti attesi.
 */
public class DateFormatException extends RuntimeException {
    /**
     * Crea l'eccezione con messaggio standard per data non valida.
     */
    public DateFormatException() {
        super(message: "ERROR! The given date's format is not accepted.");
    }
}
```

2. **TimeFormatException**: Estende la classe nativa Exception (eccezione *checked*). Viene impiegata nella classe Time per segnalare anomalie nella configurazione oraria (es. ore o minuti). Essendo un'eccezione controllata, obbliga il programmatore a implementare una gestione esplicita tramite costrutto try-catch o clausola throws nella firma del metodo.

```
/**
 * Eccezione lanciata quando un orario non rispetta il formato o i limiti attesi.
 */
public class TimeFormatException extends Exception {
    /**
     * Crea l'eccezione con messaggio standard per orario non valido.
     */
    public TimeFormatException() {
        super(message: "L'orario inserito non è valido.");
    }
}
```

Flusso Algoritmico di Gestione nei Try-Catch

L'approccio architetturale prevede che le eccezioni vengano intercettate il più vicino possibile al punto di guasto per consentire un ripristino (*recovery*) dello stato o, nel caso della CLI, la ripetizione dell'inserimento da parte dell'utente senza interrompere l'esecuzione.

Un esempio significativo è riscontrabile nel metodo di caricamento dei dati all'interno della classe Manager:

```
private Prenotazione parsePrenotazioneFromCsv(String line) {
    try {
        String[] fields = splitCsv(line);
        if (fields.length < 10) {
            return null;
        }
        String titolo = unquote(fields[4]);
        int giorno = Integer.parseInt(fields[5]);
        int mese = Integer.parseInt(fields[6]);
        int anno = Integer.parseInt(fields[7]);
        String[] oreMin = fields[8].split(regex: ":");
        Time ora = new Time(Integer.parseInt(oreMin[0]), Integer.parseInt(oreMin[1]));
        Date data = new Date(giorno, mese, anno);
        Proiezione proiezione = findProiezione(titolo, data, ora);
        LinkedList<Posto> seats = new LinkedList<>();
        if (fields.length > 10) {
            seats = parseSeats(unquote(fields[10]));
        }
        int id = Integer.parseInt(fields[0]);
        double prezzo = 0;
        if (proiezione != null) {
            prezzo = proiezione.getPrezzo() * (fields.length > 9 ? Integer.parseInt(fields[9])
                : 0);
        }
        return new Prenotazione(id, proiezione, seats, prezzo);
    } catch (DateFormatException | TimeFormatException | NumberFormatException e) {
        System.out.println("Errore durante il parsing della prenotazione: " + e.getMessage());
        return null;
    }
}
```


Gestione degli Errori di Input Oltre i Limiti Fisici della Sala

Oltre alle eccezioni basate su classi, il sistema adotta tecniche di *Defensive Programming* per intercettare violazioni dei vincoli di dominio, in particolare nel modulo di allocazione dei posti gestito dai clienti e dai bigliettai.

Prima di modificare la matrice sala, il metodo di prenotazione verifica preventivamente che l'indice calcolato sia geometricamente compatibile con i confini dell'array bidimensionale [10x20], evitando il sollevamento automatico dell'eccezione nativa della JVM `ArrayIndexOutOfBoundsException`.

```
private boolean isValidSeat(int row, int col) {  
    return row >= 1 && row <= sala.length && col >= 1 && col <= sala[0].length;  
}
```

Qualora un utente tenti di digitare un codice fila inesistente (es. "Z") o un numero di posto fuori scala (es. 25), la richiesta viene respinta in modo controllato, preservando l'integrità della memoria.

SITOGRAFIA E BIBLIOGRAFIA

Questo manuale raccoglie esclusivamente il materiale consultato per acquisire le conoscenze necessarie alla realizzazione del programma.

Materiale Didattico Universitario (Bibliografia di Riferimento)

- **Rizzardi, A. (2025).** *Slide e Dispense del Corso di Programmazione*. Università degli Studi dell'Insubria, Dipartimento di Scienze Teoriche e Applicate (DISTA).
 - *Riferimento nel progetto:* Fondamenti del paradigma Object-Oriented applicati all'architettura di CineMax. Nello specifico:
 - Regole di scomposizione in classi ed ereditarietà per la gerarchia Utente, Cliente, Bigliettaio e Proiezionista.
 - Politiche di incapsulamento rigido delle variabili di stato tramite l'uso di modificatori d'accesso (`private`, `protected`) e metodi di accesso/modifica (`getter` e `setter`).
 - Teoria e implementazione dei blocchi di gestione delle anomalie tramite costrutti nativi try-catch ed eccezioni personalizzate (`checked` e `unchecked`).
 - Utilizzo dei tipi enumerativi (`enum`) per la definizione di insiemi di costanti tipizzate (Ruolo).
- **Meroni, G. (2025).** *Slide e Dispense del Corso di Laboratorio Interdisciplinare A*. Università degli Studi dell'Insubria, Dipartimento di Scienze Teoriche e Applicate (DISTA).
 - *Riferimento nel progetto:* Linee guida strutturali, vincoli di progetto per l'organizzazione modulare dell'applicativo, specifiche per i dataset di test e direttive formali per la stesura della documentazione tecnica e dei commenti in formato JavaDoc.



CineMax

All'Avanguardia del Cinema